# 论文-44 · 门控与稀疏执行：一组工程观察（汇总版·待复核）

何淇 · 与 DeepSeek 协作

数字生命系列 · 执行器工程（汇总篇，内部编号第 44 篇）

日期：2026-11-01

状态：工程观察汇总——单种子为主、数据与算力有限；结论保守，不作方法宣称。

说明：本篇将此前分散的执行器工程观察（论文-39、40、41、42 及未发布的 43）合并为一篇。

## 摘要

执行器工程研究的是一个工程问题：**能不能让模型在推理时"少算"，而质量不明显下降？**

我们的回答是：能，但有明确边界。主要观察如下：

1. **门控训练可用于从零训练的模型**：每层一个内容门 σ(W·h+b)，用拉格朗日乘子把平均开启层数压到目标。在 GPT-2 small（12 层）上，门控模型平均使用 7.48 层，PPL 12.03（plain 对照 16.35，plain 未收敛，不作"门控更优"声称）。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**：只算门开 token 的 Q、注意力输出行与 MLP，K/V 全保留；实测 PPL 12.032 vs 12.033（无损）。这证明门控本身不伤质量，质量差异来自"执行方式"。

3. **推理端转 MoD 容量路由，质量几乎无损**：门开优先 + 每层固定容量 K_max + padding，factor=1.0 时名义省 37.9% 计算，PPL 12.107（+0.6%）。

4. **真实加速未取得**：自写 gather/索引实现下，batch 32 墙钟 0.89~1.01x；早期一次实现曾测到 1.22x，说明加速高度依赖实现，非本文可结论。

5. **注意力子集化（跳过 token 不产生 K/V）代价明显**：在已训练模型上 +6.8% PPL；从零训练两次尝试均崩（PPL 350.99 / 393.71）。

6. **规模与训练方式的影响**：从零训练时，1.2M、124M、1.5B 模型都能以约 44%~50% 的层数工作，代价 1%~7%；而预训练模型 + LoRA 后训练（3B/7B）层数压不动、PPL 反升。关键变量是训练方式，不是参数量。

## 一、问题与背景

### **1.1** 为什么关心"少算"

Transformer 每生成一个 token 都要过完所有层。若某些 token 或某些层对该 token 而言是冗余的，跳过它们就能省算力。这条思路在文献中有多个版本：MoD（Mixture-of-Depths，容量路由）、早退（early exit）、条件计算（conditional computation）、稀疏专家（MoE）等。

我们关心的是其中的"层门控"：**每层、每个 token 一个开/关决策，决定这一层是否对该 token 施加变换。**

### **1.2** 我们的术语（先定义，后文都用这个含义）

- **内容门**：每层一个线性层 W_l，输入该 token 当前的隐藏状态 h，输出 $g = \sigma(W_l \cdot h + b)$。因此门是"每层一个 W_l，但每个 token 每层各有决策"；token 经过每层的矩阵变换后 h 改变，下一层的门分数也随之改变。

- **门开（$p > 0.5$）**：这一层对该 token 计算并写回。

- **门关（$p \leq 0.5$）**：跳过这一层，该 token 的表示保持残差直通。

- **K/V**：注意力的记忆，所有 token 都产生并保留（我们的"保留 K/V"版本）。

- **Q 侧更新**：每个 token 自己的查询与注意力输出；门关时这一部分被跳过。

- **稀疏执行**：按门的决策，只对门开 token 计算 Q/注意力输出/MLP，而 K/V 始终全保留。

- **容量路由（MoD 风格）**：每层固定容量 K，按门分数取 top-K 个 token 进入该层计算（跨层容量不同）。

- **子集化**：更强的一种稀疏——被跳过的 token 不产生 K/V，注意力只在被选中的子集内计算。

### **1.3** 与已有工作的关系

我们不声称提出新方法。门控、稀疏执行、容量路由与早退同属条件计算一类；MoD 已报告过容量路由的可行性。我们的工作是：在有限的算力与数据条件下，把"门控训练 + 推理端稀疏执行"这条工程路线走一遍，记录哪些可行、哪些不可行、边界在哪里。

## 二、方法

### **2.1** 内容门 + 拉格朗日预算

每层一个内容门。训练时用拉格朗日乘子 $\lambda$ 控制平均开启比例：损失 = 语言建模损失 + $\lambda \cdot$ 平均活跃层数；$\lambda$ 按"当前平均活跃 − 目标"自适应调整。这样门不需要人工指定"哪层该关"，由训练自己学。

**2.2** 两种执行方式

- **保留 K/V 的稀疏执行**：K/V 全算；门开 token 的 Q/注意力输出/MLP 才算。

- **容量路由（MoD 风格）**：每层固定容量 K，按分数选 top-K token；其余 token 走残差。为保证形状静态，用 padding 席位固定 K_max，门开优先填满，不足的空位只算不写回。

**2.3** 实验设置（统一说明）

- 数据：tinyshakespeare（约 1MB）或 TinyStories 切片（20MB/100MB）。

- 评测：验证集 PPL；以 best 记录（早停，避免过拟合干扰）；单种子。

- 硬件：阿里云 PAI-DSW，单卡 V100 16GB 或 A10 24GB。

## 三、实验与结果

**3.1** 小模型（**6** 层，字符级，约 **1.2M** 参数）

早期实验，从头训练：

- plain 3 种子均值 PPL 6.103（6.232 / 5.999 / 6.083）；

- 内容门 + 拉格朗日目标 3 层：PPL 6.028，平均 2.97/6 层（省约一半）；

- 位置门（每层×位置固定表）：目标 3 层 → PPL 6.038，平均 2.98 层；

- λ=0 自由训练时，24 层小模型自己选择 5.3/24 层。

**观察**：小模型上门控可以用一半层数而不降质量；门是自己学出来的，不是人工指定。

**3.2 GPT-2 small**（**12** 层，**124M**），从零训练

**1MB 莎士比亚：**

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 44.03 | 1500 | 12 |

| joint_gate | 47.01 | 1000 | 5.30（44%） |

代价 +6.8%。门控在 λ≈0 时自己收敛到约 5.3 层。

**100MB TinyStories：**

| 组 | best PPL | 步数 | 平均层数 |

|---|---|---|---|

| plain | 4.892 | 9000 | 12 |

| joint_gate | 5.023 | 9000 | 5.81（48%） |

代价 +2.7%。每层活跃率（final）：`1.00, 0.82, 0.61, 0.01, 0.12, 0.35, 0.28, 0.25, 0.25, 0.45, 0.75, 0.74`。第 0 层全开、第 3 层几乎不用、末两层回到 0.74~0.75。

**观察**：数据变多（1MB → 100MB）时，层的使用比例几乎不变（56% → 52%），但代价变小（6.8% → 2.7%）。也就是说，数据多让门控"省得起"，没有让门控"省更多"。

**3.3 Qwen2.5-1.5B（28 层），从零训练，100MB**

| 组 | final PPL | best PPL | 平均层数 |

|---|---|---|---|

| plain | 6.301 | 6.501 @6500 | 28 |

| joint_gate | 6.373 | 6.113 @6500 | 12.32（44%） |

final 口径代价 +1.1%；best 口径门控组反而更低。每层活跃率显示**最后 9 层（L19~L27）几乎全部关闭**。

**观察**：模型放大到 1.5B，门控依然便宜（44% 层数、+1.1%）。这与"越大的模型越难压缩"的直觉相反，也说明关键变量是"是否从头训练"。

**3.4 预训练模型 + LoRA 后训练：压不动**

**3B（36 层）与 7B（28 层）**，加载预训练权重，LoRA r=16，数据 1MB，2000 步：

| 设置 | plain best | 门控 best | 用的层 | 代价 |

|---|---|---|---|---|

| Qwen2.5-3B 目标 33 | 12.61 | 14.94 | 32.79/36（91%） | +18% |

| Qwen2.5-7B 目标 26 | 7.01 | 9.71 | 25.70/28（92%） | +38% |

| Qwen2.5-7B 目标 24 | 7.01 | 12.93 | 24.01/28（86%） | +85% |

| Qwen2.5-7B 目标 22 | 7.01 | 12.79 | 22.27/28（80%） | +82% |

7B 目标 26 组后段还发生门控塌缩（PPL 升到 1243，层数掉到 20）。

**观察**：预训练权重 + 轻量后训练 + 小数据，层数压不动、质量反升。这不构成"大模型不能压缩"的结论，只能说明"这种后训练方式不行"。

**3.5 GPT-2** 门控模型 **+** 推理端 **MoD**（容量路由）

在 3.2 的 GPT-2 门控模型（7.48/12 层，PPL 12.03）上测试多种推理方式：

| 推理方式 | PPL | 相对 dense | 平均活跃 | 说明 |

|---|---|---|---|---|

| dense gated（参照） | 12.033 | — | — | 全部计算 |

| open_only（只算门开 token，K/V 全留） | **12.032** | **等价** | 0.622 | 动态形状 |

| open_cap（门开优先 + 容量上限） | 12.176 | +1.2% | 0.592 | 动态形状 |

| global top-k（全局 top-k） | 12.182 | +1.2% | 0.624 | 动态形状 |

| static padded（K_max + mask） | 12.046~12.107 | +0.1%~+0.6% | 0.62~0.77 | 静态形状 |

**K_max 余量扫描**（dense 参照 12.034；K_max = 平均门开数 × factor）：

| factor | PPL | 平均活跃 | 名义省算力 |

|---|---|---|---|

| 1.35 | 12.046 | 0.767 | 23.3% |

| 1.25 | 12.054 | 0.730 | 27.0% |

| 1.15 | 12.051 | 0.691 | 30.9% |

| 1.05 | 12.098 | 0.649 | 35.1% |

| **1.00** | **12.107** | **0.621** | **37.9%** |

| 0.95 | 14.458 | 0.592 | 40.8% |

**观察**：质量拐点在 factor 1.0~1.05；质量优先的落点是 factor=1.0，名义省 37.9%、PPL +0.6%。

**真实墙钟（重要负结果）**：

| 实现 | batch | dense | 稀疏 | 加速 |

|---|---|---|---|---|

| 早期 query 端稀疏 | 1 | 18.9 ms | 20.5 ms | 0.93x |

| 同上 | 8 | 19.7 ms | 20.8 ms | 0.95x |

| 同上 | 32 | 43.4 ms | 35.5 ms | 1.22x |

| 静态填充实现 | 32 | 38.9 ms | 41.7~43.4 ms | 0.89~0.93x |

| 静态填充实现 | 64 | 73.9 ms | 73.5 ms | 1.01x |

自写实现是 Python 级 gather + 自定义注意力，远慢于 PyTorch 融合 kernel。早期 1.22x 与后期 0.89x 的差异说明：**墙钟结果高度依赖实现细节，不能作为稳定结论**。

**3.6** 注意力子集化：失败

**(a) 在已训练模型上直接切换子集化语义**（跳过 token 不产生 K/V）：

| 容量 | PPL | 相对 dense |

|---|---|---|

| quantized（名义省 37.5%） | 12.855 | +6.8% |

| half（45.8%） | 14.030 | +16.6% |

| quarter（68.75%） | 18.692 | +55% |

**(b) 从零训练子集化（两次尝试）**：

| 尝试 | plain PPL | 子集化 PPL | 活跃率 |

|---|---|---|---|

| 门 + 阈值式动态子集 | 19.20 | 350.99 | 塌到 8.3% |

| 门当路由器 + 固定容量 top-k + 课程 | 19.03 | 393.71 | 0.496（容量未塌） |

**(c) 短微调恢复**：400 步、lr 5e-5，PPL 崩到 136（超参不当，不能作为"微调无效"的证据）。

**观察**：注意力子集化在我们这套门控路由上代价明显；从零训练更差。但这不是"子集化本身不可行"的结论——MoD 原方法用独立路由器 + 容量路由，在更大规模上验证过该语义。

### **3.7** 分支独立门控：失败

每层两个门（注意力门、MLP 门分开），从零训练 12000 步：final PPL 14.484（注意力门 0.61、MLP 门 0.47）；作为对照，联合门控（一个门管两分支）4000 步就到 12.033。分支独立门控即使训练 3 倍步数仍明显更差。

**分支稀疏拆分评测**（在分支独立门控模型上，用 top-k 强制压低单边）：

| 测试 | PPL |

|---|---|

| baseline | 14.486 |

| MLP 压到 25% | 558.95 |

| 注意力压到 25% | 69.45 |

| 两边都压 25% | 2195.20 |

**观察**：在这个模型上，"注意力更新"比"MLP 更新"更耐稀疏。结合"保留 K/V 无损、子集化伤质量"的结果，初步图景是：**MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省**。但这只是单模型观察，不能当结论。

## 四、讨论

### **4.1** 为什么**"保留 K/V"**明显更好

**[解释]** 注意力是跨 token 的信息通路。被跳过的 token 若从注意力中消失，后续 token 少看到一部分上下文，语言建模难以补偿。门控只决定"是否更新该 token 的表示"；保留 K/V 时，更新与否不影响其他 token 可访问的信息。这是我们看到"保留 K/V 无损、子集化 +6.8%"的直接原因。

### **4.2** 为什么从零训练与后训练差别这么大

**[解释]** 从零训练时，模型从一开始就在"可能跳层"的约束下学习，可以把计算集中到少数层（1.5B 的最后 9 层几乎全关就是证据）。预训练模型没有这个机会：每一层都被预训练"用惯了"，LoRA 低秩更新改不动这个结构。

**4.3** 名义省算力 **vs** 真实加速

**[事实]** 我们测到的是名义计算量（层数×token 比例）。真实墙钟取决于 kernel 实现、批大小、内存访问；我们自写的实现在 batch 32/64 上为 0.89~1.22x，不能作为稳定结论。真实加速是独立的工程问题，超出本实验条件。

## 五、局限

- 单种子为主；数据规模有限（1MB~100MB）；plain 在部分实验中未收敛；

- 从零训练组的绝对 PPL 不代表模型真实能力（欠训练）；

- 仅 PPL，无下游任务；

- 墙钟测量受实现影响大；

- 未与 MoD 原文做同设置对比；

- 未验证 1B 以上从零训练、未验证跨域泛化。

## 六、结论

1. **门控训练可用**，且门是"每层一个 W_l、每 token 每层一个决策、跟着 token 表示走"的内容门。

2. **"保留 K/V 的稀疏执行"与稠密门控精确等价**——门控本身不伤质量；质量差异来自执行方式。

3. **推理端容量路由（门开优先 + 静态 K_max）可做到名义省 37.9%、PPL +0.6%**；质量拐点在 K_max ≈ 平均门开数。

4. **注意力子集化在我们这套门控路由上代价明显**（+6.8%；从零训练崩），不推广为子集化本身不可行。

5. **关键变量是训练方式而非参数量**：从零训练时 1.2M~1.5B 都能省一半左右；预训练 + LoRA 后训练压不动。

6. **真实加速是独立的工程问题**，本文不作结论。

## 附：标注

- **[事实]** 第三节全部 PPL、层数、活跃率、墙钟（来自 results JSON）。

- **[解释]** 第四节的分析。

- **[推测]** "从零训练会把计算集中到少数层"——由 1.5B 层活跃率支持，未在更大规模验证。

- **[不声称]** 不声称新方法；不声称真实加速；不声称与 MoD 原文可比。

- **旧篇关系**：本篇合并论文-39、40、41、42 及未发布的 43；旧篇保留 DOI，视为被本篇取代（superseded）。